CS1004 – Object Oriented Programming

Lecture # 20
Tuesday, May 13, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

Muhammad Yousaf

Operator overloading and Assignment

- Assignment between types is handled by compiler
- For native datatypes

```
int num1, num2=50;
num1 = num2;
```

- And for user defined data types

```
Distance dist1, dist2,dist3;
dist3 = dist1 + dist2;
```

Operator overloading and type conversion

- what happens when the variables on different sides of the = are of different type
  - Implicit type conversion a.k.a. type coercion

```
int intVal;
float floatVal;
intVal = floatVal;
```

- We assume that compiler calls a special routine to convert from float stored in `floatVal` to and integer to be stored in `intVal`
- We can also explicitly convert one datatype to other

```
intVal = static_cast<int>(floatVal);
```

Conversion between objects and built-in datatypes

- We cannot rely on compiler
  - It does not know about user defined datatypes
  - We need to provide explicit type conversion method

```
const float MTF; //meters to feet
Distance(float meters) : MTF(3.280833F){
//convert meters to Distance
     float fltfeet = MTF * meters;
     feet = int(fltfeet);
     inches = 12 * (fltfeet - feet);
}
operator float() const //conversion operator
{ //converts Distance to meters
     float fracfeet = inches / 12;
     //convert the inches
     fracfeet += static_cast<float>(feet);
     //add the feet
     return fracfeet / MTF;
}
```

Calls float conversion operator

Calls one argument constructor

```
int main() //For type conversion test
{
     float mtrs;
     Distance dist1 = 2.35F; //uses 1-arg
     //constructor to convert meters to Distance
     cout << "\ndist1 = "; dist1.showdist();
     mtrs = static_cast<float>(dist1); //uses
     //conversion operator for Distance to meter
     cout << "\ndist1 = " << mtrs
          << " meters\n";
     Distance dist2(5, 10.25);
     //uses 2-arg constructor
     mtrs = dist2; //also uses conversion op
     cout << "\ndist2 = " << mtrs
     << " meters\n";
     // dist2 = mtrs; //error, = won't convert
return 0;
}
```

Conversion between objects of different classes

- We can use
  - A one argument constructor
  - A conversion operator

```
A objA; //Object of Class A
B objB; //Object of Class B
objA = objB;
```

- Conversion routine can be a part of
  - Source class (conversion operator)
  - Destination class (one argument constructor)

Converting object of one class to other using conversion

```
class time12 {
private:
     bool pm;
     int hrs, mins;
public:
     time12():pm(false), hrs(12), mins(0) {};
     time12(bool ap, int h, int m) :pm(ap), hrs(h),
                                   mins(m) {}
     void display()const
     {
          cout << hrs << ':';
          if (mins < 10)
               cout << '0';
          cout << mins << ' ';
          cout << (pm ? "p.m." : "a.m.");
     }
};
```

```
class time24
{
private:
     int hours;
     int minutes;
     int seconds;
public:
     time24() :hours(0), minutes(0), seconds(0) {}
     time24(int h, int m, int s) :hours(h),
                                  minutes(m), seconds(s) {}
     void display()const
     {
          if (hours < 10) cout << '0';
          cout << hours << ':';
          if (minutes < 10) cout << '0';
          cout << minutes << ':';
          if (seconds < 10) cout << '0';
          cout << seconds;
     }
     operator time12() const;
};
```

Converting object of one class to other using conversion

```
time24::operator time12() const{
      int hrs24 = hours;
      bool pm = hours < 12 ? false : true;
      int roundMins = seconds<30 ? minutes:minutes + 1;
      if (roundMins == 60) //carry mins?{
            roundMins = 0;
            ++hrs24;
            if (hrs24 == 12 || hrs24 == 24)
            //carry hours?
            pm = (pm == true) ? false : true;
            //toggle am/pm
      }
      int hrs12 = (hrs24 < 13) ? hrs24 : hrs24 - 12;
      if (hrs12 == 0) //00 is 12 a.m.{
            hrs12 = 12;
            pm = false;
      }
      return time12(pm, hrs12, roundMins);
}
```

- Conversion function converts military format to a 12-hour format and returns an object of 12-hour format.

```
int main()
{
      int h, m, s;
      cout << "Enter 24 - hour time : \n";
      cout << " Hours(0 to 23) : "; cin >> h;
      cout << " Minutes : "; cin >> m;
      cout << " Seconds : "; cin >> s;
      time24 t24(h, m, s); //make a time24
      cout << "You entered : "; //display the
                        //time24
      t24.display();
      time12 t12 = t24; //convert time24 to
                        //time12
      cout << "\n12 - hour time : "; //display
                        //equivalent time12
      t12.display();
      cout << "\n\n";
      return 0;
}
```

```
time12 t12 = t24;
```

- The above mentioned instruction converts the time in military format to civilian format. While the conversion function is part of source class

Conversion using destination class

```
class time24{
      int hours; //0 to 23
      int minutes; //0 to 59
      int seconds; //0 to 59
public: //no-arg constructor
      time24() : hours(0), minutes(0), seconds(0) { }
      time24(int h, int m, int s) : //3-arg constructor
                  hours(h), minutes(m), seconds(s) { }
      void display() const //format 23:15:01
      {
            if (hours < 10) cout << '0';
            cout << hours << ':';
            if (minutes < 10) cout << '0';
            cout << minutes << ':';
            if (seconds < 10) cout << '0';
            cout << seconds;
      }
      int getHrs() const { return hours; }
      int getMins() const { return minutes; }
      int getSecs() const { return seconds; }
};
```

```
class time12
{
      bool pm; //true = pm, false = am
      int hrs; //1 to 12
      int mins; //0 to 59
public: //no-arg constructor
      time12() : pm(true), hrs(0), mins(0) { }
      time12(time24); //1-arg constructor
      time12(bool ap, int h, int m) :
                  pm(ap),hrs(h),mins(m){ }
      void display() const
      {
            cout << hrs << ':';
            if (mins < 10) cout << '0';
            cout << mins << ' ';
            string am_pm = pm ? "p.m." : "a.m.";
            cout << am_pm;
      }
};
```

Conversion using destination class

```
time12::time12(time24 t24) //1-arg constructor
{ //converts time24 to time12
     int hrs24 = t24.getHrs(); //get hours
     pm = t24.getHrs() < 12 ? false : true;
     mins = (t24.getSecs() < 30) ? //round secs
     t24.getMins() : t24.getMins() + 1;
     if (mins == 60) //carry mins?
     {
          mins = 0;
          ++hrs24;
          if (hrs24 == 12 || hrs24 == 24)
          pm = (pm == true) ? false : true;
     }
     hrs = (hrs24 < 13) ? hrs24 : hrs24 - 12;
     if (hrs == 0) //00 is 12 a.m.
          { hrs = 12; pm = false; }
}
```

```
int main()
{
     int h, m, s;
     cout << "Enter 24 - hour time : \n";
     cout << " Hours(0 to 23) : "; cin >> h;
     if (h > 23) //quit if hours > 23
          return(1);
     cout << " Minutes : "; cin >> m;
     cout << " Seconds : "; cin >> s;
     time24 t24(h, m, s); //make a time24
     cout << "You entered : ";
     t24.display();

     time12 t12 = t24; //convert time24 to
time12

     cout << "\n12 - hour time : ";
     t12.display();
     cout << "\n\n";
     return 0;
}
```

Here constructor of time12 will be called for conversion

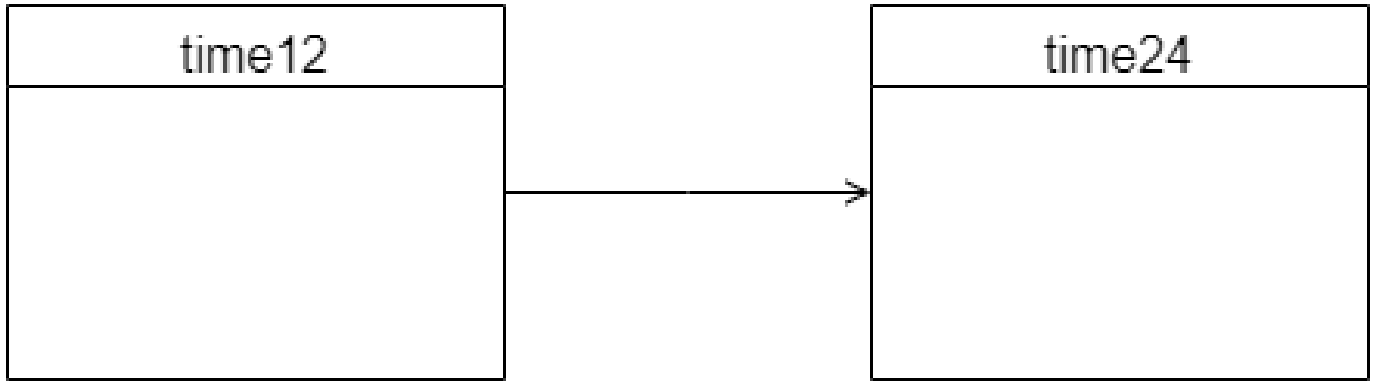
When to use what

- When to use
  - one-argument constructor in the destination class

    OR
  - conversion operator in the source class
- If we do not have access to the code of one class i.e. a purchased library
  - If we use object of such a class as source, then we have to use one argument constructor
  - If we use object of such a class as destination, then we have to use conversion operator in source

UML class diagram

- Association
  - A class association actually implies that objects of the classes, rather than the classes themselves, have some kind of relationship



Not all operator can be overloaded

- The member access or dot operator (.)
- The scope resolution operator (::)
- The conditional operator (?:)
- The pointer-to-member operator (->)
- **Note**
  - New operator cannot be created or overloaded

Friend functions and overloading

- friends are used to increase the versatility of overloaded operators
- The class distance with one argument constructor and overloaded + operator we can have the below mentioned instructions

```
Distance d1 = 2.5, Distance d2 = 1.25, d3; //constructor converts float feet to Distance
d3 = d1 + d2; //Calls operator + overloaded
d3 = d1 + 10.0; //distance + float: OK
```

- The above instruction calls single argument constructor to convert 10.0 to object of Distance and the calls operator + overloaded function

```
d3 = 10.0 + d1; //float + Distance: ERROR
```

- The above line generates error as 10.0 is a float and there is no conversion available in default float that can convert a Distance object to float

Solution to the conversion

- The solution to the problem of conversion mentioned earlier is to make an explicit object of Distance

```
d3 = Distance(10.0) + d1;
```

- It is nonintuitive and inelegant
- To get around this problem **friend function** can provide the solution

Operator + using friend function

```
class Distance {
     ...
public:
     ...
     friend Distance operator + (Distance, Distance); //friend
};
-----------------------------------
Distance operator + (Distance d1, Distance d2) //add d1 to d2
{
     int f = d1.feet + d2.feet; //add the feet
     float i = d1.inches + d2.inches; //add the inches
     if (i >= 12.0) //if inches exceeds 12.0,
     {
          i -= 12.0; f++;
     } //less 12 inches, plus 1 foot
     return Distance(f, i); //return new Distance with sum
}
```

- Now the instruction d3 = 10.0 + d1; will execute without error as now it will fiend a function with two Distance arguments. So it will convert first 10.0 to a Distance object

Questions